

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине «Алгоритмы и структуры данных»
Тема: «Дерево отрезков с массовыми операциями»

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

СОДЕРЖАНИЕ

1.	Цель работы	3
2.	Задание	4
3.	Теоретический минимум	5
4.	Выполнение работы	7
4.1.	Описание структуры кода	7
4.2.	Описание пайплайна	8
5.	Исследование	9
6.	Вывод	10
	Приложение А. Код программы	11
	Приложение Б. Тестирование	13

1. ЦЕЛЬ РАБОТЫ

Реализовать структуру данных дерево отрезков, которая будет поддерживать массовую операцию сложения и операцию получения элемента по индексу. Исследовать асимптотическую сложность функций и показать на графике зависимость времени работы от размера введенных данных.

2. ЗАДАНИЕ

Есть массив из n элементов, изначально заполненный нулями. Вам нужно написать структуру данных, которая обрабатывает два вида запросов:

- прибавить к отрезку от l до $r-1$ число v
- узнать текущее значение в ячейке i

Входные данные

Первая строка содержит два числа n и m ($1 \leq n \leq 10^4$, $1 \leq m \leq 10^4$) — размер массива и число операций.

Далее следует описание операций. Описание каждой операции имеет следующий вид:

- $1\ l\ r\ v$ — прибавить значение v к отрезку от l до $r-1$ ($0 \leq l < r \leq n$, $0 \leq v \leq 10^9$).
- $2\ i$ — найти значение в ячейке с индексом i ($0 \leq i < n$).

Выходные данные

Для каждой операции второго типа выведите соответствующее значение.

3. ТЕОРЕТИЧЕСКИЙ МИНИМУМ

Дерево отрезков — структура данных, предназначенная для эффективной обработки запросов на отрезках массива. Оно позволяет выполнять операции обновления и получения агрегированной информации за логарифмическое время.

Дерево строится над массивом путём рекурсивного разбиения диапазона на два подотрезка. Каждая вершина хранит агрегированное значение для своего сегмента (например, сумму или минимум). Корень дерева соответствует всему диапазону.

Асимптотика:

- построение — $O(n)$,
- запрос/обновление — $O(\log n)$.

Массовыми называются операции, затрагивающие все элементы некоторого диапазона. Прямая обработка каждого элемента приведёт к сложности $O(n)$, поэтому применяется механизм ленивого распространения.

Для каждой вершины хранится не только текущее агрегированное значение, но и отложенная операция. При выполнении массового обновления операция записывается в lazy-метку без немедленного спуска к потомкам. При обращении к потомкам отложенная операция применяется к значению вершины, корректно комбинируется с существующими lazy-метками, затем передаётся дочерним вершинам. Таким образом обеспечивается $O(\log n)$ на каждое массовое обновление и запрос.

Операция обновления диапазона:

Если обновляемый отрезок полностью покрывает текущую вершину, устанавливается соответствующая lazy-метка и обновляется агрегированное значение. В противном случае метки спускаются, и операция распределяется по детям.

Операция запроса на диапазоне:

Перед обработкой запроса ленивые метки спускаются. Если запрашиваемый отрезок полностью покрывает сегмент вершины, возвращается её значение; при частичном покрытии данные объединяются из поддеревьев.

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1. Описание структуры кода

Реализована структура *Node*, хранящая информацию об отдельном узле дерева отрезков.

Поле структуры:

- *long long d* — lazy-значение узла.

Для удобной работы с деревом создан класс *SegmentTree*.

Поле класса:

- *std::vector<Node> tree* — массив, в котором последовательно хранятся узлы дерева.

Методы класса:

1. *SegmentTree(long long n)* — конструктор, создающий полное двоичное дерево размера $4*n$ (где n — количество элементов массива) и инициализирующий все lazy-значения нулями.
2. *void push(long long node)* — метод «проталкивания» lazy-значения от родителя к детям.

Принимаемый аргумент:

long long node — узел-родитель.

3. *void update(long long node, long long l, long long r, long long ql, long long qr, long long v)* — метод массовой операции сложения на заданном отрезке.

Принимаемые аргументы:

long long node — индекс узла, с которого начнется обновление.

long long l — индекс левой границы массива.

long long r — индекс правой границы массива.

long long ql — индекс левой границы обновляемого отрезка.

long long qr — индекс правой границы обновляемого отрезка.

long long v — значение, которое будет прибавлено ко всем элементам на заданном отрезке.

4. *long long getValue(long long node, long long l, long long r, long long index)*
— метод получения значения по индексу.

Принимаемые аргументы:

long long node — узел, в котором происходит поиск.

long long l — левая граница поиска.

long long r — правая граница поиска.

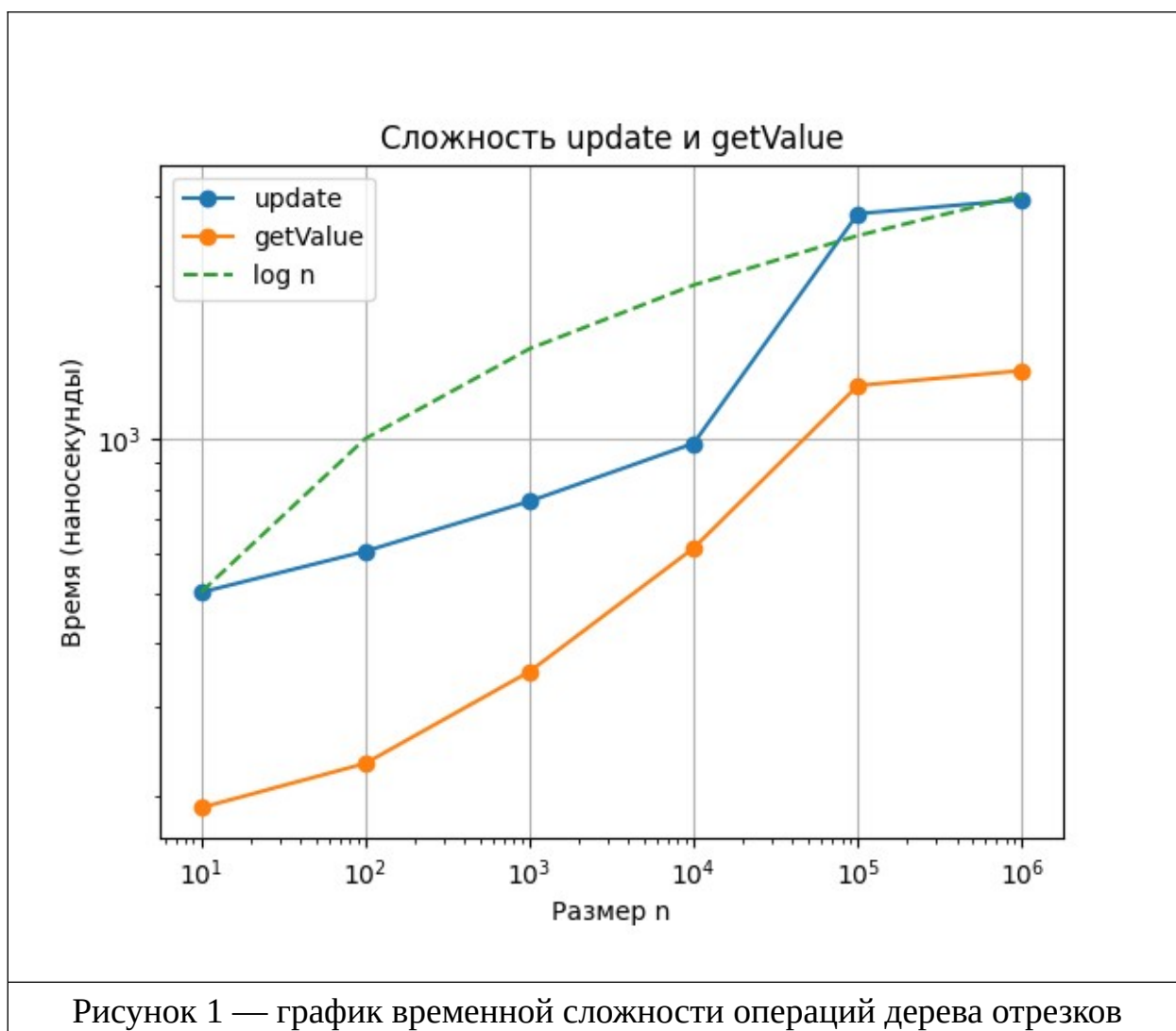
long long index — индекс искомого элемента.

4.2. Описание пайплайна

Функция *main* предназначена для работы с данными. Считывание происходит через стандартный поток ввода. По полученному размеру массива строится дерево отрезков, к которому затем применяются *m* операций массового обновления и поиска по индексу (в зависимости от введенных параметров). Для хранения результатов поиска создан массив *ans*. После выполнения всех операций с помощью цикла происходит вывод значений из массива *ans*.

ИССЛЕДОВАНИЕ

Для оценки временной сложности операций структуры данных дерева отрезков было проведено исследование времени выполнения функций массового обновления и получения элемента по индексу при различных объёмах данных (от 10 до 10^6 элементов). По результатам эксперимента построены графики зависимости времени выполнения от количества узлов (см. рис. 1). Время выполнения операций *update* и *getValue* растёт примерно логарифмически от размера данных, что соответствует теоретической сложности $O(\log n)$.



6. ВЫВОД

В результате выполнения лабораторной работы была изучена структура данных дерево отрезков, разработаны методы массового обновления элементов на заданном отрезке и поиска элемента по индексу. Проведено экспериментальное исследование временной сложности реализованных операций на различных размерах массивов и построены соответствующие графики, демонстрирующие зависимость времени работы от размера входных данных. Графики подтвердили теоретические оценки асимптотической сложности: операции обновления диапазона и получения значения выполняются за логарифмическое время, что соответствует сложности $O(\log n)$. Произведено тестирование программы, подтверждающее корректность ее работы и соответствие ее поведения теоретической модели.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
#include <iostream>
#include <vector>

struct Node{
    long long d;
};

class SegmentTree{
    std::vector<Node> tree;
public:

    SegmentTree(long long n){
        tree.resize(4*n, {0});
    }

    void push(long long node){
        if (tree[node].d != 0) {
            tree[node*2+1].d += tree[node].d;
            tree[node*2+2].d += tree[node].d;
            tree[node].d = 0;
        }
    }

    void update(long long node, long long l, long long r, long long
ql, long long qr, long long v){
        if (qr < l || r < ql) return;

        if (ql <= l && r <= qr){
            tree[node].d += v;
            return;
        }

        push(node);

        long long mid = (l + r) / 2;
        update(node*2+1, l, mid, ql, qr, v);
        update(node*2+2, mid+1, r, ql, qr, v);
    }

    long long getValue(long long node, long long l, long long r, long
long index){
        if (l == r){
            return tree[node].d;
        }

        push(node);

        long long mid = (l + r) / 2;
        if (index <= mid)
            return getValue(node*2+1, l, mid, index);
        else
            return getValue(node*2+2, mid+1, r, index);
    }
};
```

```

    }
};

int main(){
    long long n, m;
    std::cin >> n >> m;

    SegmentTree st(n);
    std::vector<long long> ans;

    for (int j = 0; j < m; j++){
        int type;
        std::cin >> type;

        if (type == 1){
            long long l, r, v;
            std::cin >> l >> r >> v;
            st.update(0, 0, n-1, l, r-1, v);
        } else {
            long long i;
            std::cin >> i;
            ans.push_back(st.getValue(0, 0, n-1, i));
        }
    }

    for (auto x : ans)
        std::cout << x << "\n";

    return 0;
}

```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

```
#include <gtest/gtest.h>
#include "../main.cpp"

TEST(Test, SingleQuery) {
    SegmentTree st(5);
    st.update(0, 0, 4, 2, 2, 10);

    EXPECT_EQ(st.getValue(0, 0, 4, 2), 10);

    for (int i = 0; i < 5; ++i) {
        if (i == 2) continue;
        EXPECT_EQ(st.getValue(0, 0, 4, i), 0);
    }
}

TEST(Test, RangeUpdate) {
    SegmentTree st(6);

    st.update(0, 0, 5, 1, 4, 3);

    for (int i = 0; i < 5; ++i) {
        if (i >= 1 && i <= 4){
            EXPECT_EQ(st.getValue(0, 0, 5, i), 3);
        } else {
            EXPECT_EQ(st.getValue(0, 0, 5, i), 0);
        }
    }
}

TEST(Test, SeveralUpdates) {
    SegmentTree st(8);

    st.update(0, 0, 7, 0, 3, 5);
    st.update(0, 0, 7, 2, 6, 7);

    EXPECT_EQ(st.getValue(0, 0, 7, 0), 5);
    EXPECT_EQ(st.getValue(0, 0, 7, 1), 5);
    EXPECT_EQ(st.getValue(0, 0, 7, 2), 12);
    EXPECT_EQ(st.getValue(0, 0, 7, 3), 12);
    EXPECT_EQ(st.getValue(0, 0, 7, 4), 7);
    EXPECT_EQ(st.getValue(0, 0, 7, 5), 7);
    EXPECT_EQ(st.getValue(0, 0, 7, 6), 7);
    EXPECT_EQ(st.getValue(0, 0, 7, 7), 0);
}

TEST(Test, NoUpdates) {
    SegmentTree st(5);

    for (int i = 0; i < 5; ++i) {
        EXPECT_EQ(st.getValue(0, 0, 4, i), 0);
    }
}
```